JAVASCRIPT FUNCTIONS

Complete Methods, Properties, Code & Explanation

Beginner-to-advanced reference • Examples • Common mistakes • Practice

Inside this PDF

• Function creation styles: declaration, expression, arrow, constructor, method and generator.

• Function object methods: call(), apply(), bind(), toString(), and Symbol.hasInstance.

• Function properties: name, length, prototype, arguments and caller (legacy/deprecated notes).

• Callbacks, closures, higher-order functions, recursion, rest parameters and default parameters.

• Practical programs, interview comparisons and exercises.

Important scope: JavaScript does not provide hundreds of built-in methods on functions. The standard Function.prototype methods and related properties are covered here; common array/string methods are separate topics.

1. Function fundamentals

A **function** is a reusable block of code that performs a particular task. You define it and call it whenever you need that task.

```
function greet() {
  console.log("Hello!");
}

greet(); // Hello!
```

How it works: `function` starts the declaration, `greet` is the function name, braces contain its instructions, and `greet()` executes it.

Term	Meaning	Example
Parameter	Input variable written in definition	a, b in function add(a,b)
Argument	Actual value passed during call	add(4, 5)
Return	Sends a result back to caller	return a + b
Invocation	Executing a function	greet()

2. Ways to create functions

2.1 Function declaration

```
function add(a, b) {
  return a + b;
}
console.log(add(4, 6)); // 10
```

A named function declaration. It is hoisted, so it can normally be called before its declaration in the same scope.

2.2 Function expression

```
const multiply = function (a, b) {
  return a * b;
};
console.log(multiply(3, 4)); // 12
```

A function is stored in a variable. The variable must be initialized before it is called.

2.3 Arrow function

```
const square = n => n * n;
const add = (a, b) => a + b;
const greet = () => console.log("Hi");
```

Arrow syntax is shorter. An arrow function does not have its own `this` or `arguments`, and it cannot be called with `new`.

2.4 Function constructor

```
const add = new Function("a", "b", "return a + b");
console.log(add(2, 3)); // 5
```

Builds a function from strings. Avoid it in normal application code: it is harder to debug and can create security risks if strings contain untrusted input.

2.5 Method inside an object

```
const user = {
  name: "Aman",
  greet() {
    return "Hello " + this.name;
  }
};
console.log(user.greet()); // Hello Aman
```

A method is a function stored as an object property. Here, `this` refers to the object when called as `user.greet()`.

2.6 Generator function

```
function* makeIds() {
  yield 1;
  yield 2;
}
const ids = makeIds();
console.log(ids.next()); // { value: 1, done: false }
console.log(ids.next()); // { value: 2, done: false }
```

A generator pauses at `yield` and resumes on the next call to `next()`.

3. Function object methods (core reference)

Functions are objects in JavaScript. These methods are inherited from `Function.prototype`.

3.1 call()

Purpose: Calls a function immediately and sets the value of `this`. Arguments are passed one by one.

```
function introduce(city, country) {
  console.log(this.name + " lives in " + city + ", " + country);
}
const person = { name: "Aman" };

introduce.call(person, "Delhi", "India");
// Aman lives in Delhi, India
```

Breakdown: first argument (`person`) becomes `this`; remaining arguments are passed to the function in order.

3.2 apply()

Purpose: Like `call()`, it calls immediately and sets `this`, but arguments are supplied as an array (or array-like value).

```
function total(a, b, c) {
  return this.base + a + b + c;
}
const data = { base: 10 };

console.log(total.apply(data, [1, 2, 3])); // 16
```

Breakdown: `data` becomes `this`; the array values become the function's arguments.

3.3 bind()

Purpose: Returns a new function with `this` permanently set to the supplied value. It does not execute the function immediately.

```
const account = {
  owner: "Riya",
  showOwner() {
    return this.owner;
  }
};

const show = account.showOwner.bind(account);
console.log(show()); // Riya
```

Breakdown: `bind(account)` creates a new function. Calling `show()` later uses `account` as `this`.

3.4 toString()

Purpose: Returns a source-like string representation of a function. Exact output formatting is implementation-dependent.

```
function add(a, b) {
  return a + b;
}
console.log(add.toString());
// commonly returns the function's source text
```

Useful for inspection/debugging, not for reliably extracting or evaluating function logic.

3.5 Function.prototype[Symbol.hasInstance]

Purpose: This well-known symbol customizes how the `instanceof` operator checks whether an object is an instance of a constructor.

```
class Animal {}
const pet = new Animal();

console.log(pet instanceof Animal); // true
console.log(Animal[Symbol.hasInstance](pet)); // true
```

Most developers use `instanceof` directly. The symbol method is an advanced customization point.

4. call vs apply vs bind

Method	Executes now?	How arguments are passed?	What it returns
call()	Yes	One by one	Function's return value
apply()	Yes	As an array / array-like	Function's return value
bind()	No	Can preset arguments	A new bound function

```
function introduce(role, city) {
  return `${this.name} is a ${role} in ${city}`;
}
const person = { name: "Neha" };

introduce.call(person, "developer", "Pune");
introduce.apply(person, ["developer", "Pune"]);

const later = introduce.bind(person, "developer");
later("Pune");
```

5. Function properties

Property	What it tells you	Example
name	Function's name	add.name
length	Number of parameters before first default parameter	function f(a,b=1,c){} → f.length is 1
prototype	Prototype object used by ordinary constructible functions	Person.prototype
arguments	Legacy function-instance property; not recommended	Use rest parameters instead
caller	Legacy/deprecated and restricted in modern JS	Avoid using

5.1 name and length

```
function calculate(price, tax, discount) {
  return price + tax - discount;
}
console.log(calculate.name);   // "calculate"
console.log(calculate.length); // 3

function sample(a, b = 10, c) {}
console.log(sample.length); // 1
```

The `length` property counts formal parameters only up to the first parameter with a default value. Rest parameters are not counted.

5.2 prototype

```
function Person(name) {
  this.name = name;
}
Person.prototype.sayHi = function () {
  return "Hi, " + this.name;
};

const p1 = new Person("Aman");
console.log(p1.sayHi()); // Hi, Aman
```

For constructor functions, objects created with `new` inherit from the constructor's `prototype` object. Arrow functions do not have a normal constructor prototype and cannot be used with `new`.

6. Parameters and arguments

6.1 Default parameters

```
function welcome(name = "Guest") {
  return "Welcome " + name;
}
console.log(welcome());       // Welcome Guest
console.log(welcome("Riya")); // Welcome Riya
```

6.2 Rest parameters (...)

```
function sum(...numbers) {
  let total = 0;
  for (const n of numbers) {
    total += n;
  }
  return total;
}
console.log(sum(10, 20, 30)); // 60
```

Rest parameters collect remaining arguments into a real array. Only one rest parameter is allowed, and it must be last.

6.3 Destructured parameters

```
function showUser({ name, age }) {
  console.log(name, age);
}
showUser({ name: "Aman", age: 20 });
```

Destructuring extracts object properties into local parameter variables.

7. Callback functions

A callback is a function passed to another function so that it can be called later or at a chosen point.

```
function processUser(name, callback) {
  console.log("Processing " + name);
  callback();
}

function done() {
  console.log("Done!");
}

processUser("Aman", done);
```

Notice `done` is passed without parentheses. Writing `done()` would execute it immediately and pass its return value instead.

8. Higher-order functions

A higher-order function accepts another function as an argument, returns a function, or both.

```
function multiplier(factor) {
  return function (number) {
    return number * factor;
  };
}

const double = multiplier(2);
console.log(double(8)); // 16
```

Here, `multiplier` returns a new function. The returned function remembers `factor`.

9. Closures

A closure occurs when a function remembers and can access variables from the scope in which it was created, even after that outer function has finished.

```
function makeCounter() {
  let count = 0;

  return function () {
    count++;
    return count;
  };
}

const counter = makeCounter();
console.log(counter()); // 1
console.log(counter()); // 2
console.log(counter()); // 3
```

Why does it work? The inner function keeps access to the lexical environment containing `count`. Each call to `makeCounter()` creates a separate counter environment.

10. this keyword in functions

```
const student = {
  name: "Riya",
  show() {
    console.log(this.name);
  }
};
student.show(); // Riya

const arrowStudent = {
  name: "Riya",
  show: () => console.log(this.name)
};
arrowStudent.show(); // usually not "Riya": arrow inherits outer this
```

For ordinary functions, `this` is determined by how the function is called. Arrow functions capture `this` from their surrounding lexical scope. In strict mode, a plain function call has `this` as undefined.

11. Recursion

Recursion means a function calls itself. A correct recursive function needs a base case to stop.

```
function factorial(n) {
  if (n <= 1) return 1;       // base case
  return n * factorial(n-1); // recursive case
}

console.log(factorial(5)); // 120
```

Calculation: 5 × 4 × 3 × 2 × 1 = 120. Without a stopping condition, recursion can continue until a stack overflow.

12. Immediately Invoked Function Expression (IIFE)

```
(function () {
  const message = "Runs immediately";
  console.log(message);
})();

// Arrow IIFE
(() => {
  console.log("Arrow IIFE");
})();
```

An IIFE is a function expression that is invoked immediately. It can create a private local scope.

13. Async functions and promises

13.1 Promise-returning function

```
function getNumber() {
  return Promise.resolve(42);
}
```

```
getNumber().then(value => console.log(value)); // 42
```

13.2 async / await

```
async function loadData() {
  try {
    const response = await fetch("https://jsonplaceholder.typicode.com/todos/1");
    if (!response.ok) throw new Error("Request failed");
    const data = await response.json();
    console.log(data);
  } catch (error) {
    console.error(error);
  }
}
loadData();
```

An `async` function always returns a Promise. `await` pauses that async function until the awaited Promise settles; it does not block the whole JavaScript thread.

14. Practical function patterns

14.1 Validate input

```
function isAdult(age) {
  if (typeof age !== "number" || Number.isNaN(age)) {
    return false;
  }
  return age >= 18;
}
console.log(isAdult(20)); // true
console.log(isAdult("20")); // false
```

14.2 Function returning an object

```
function createUser(name, age) {
  return {
    name: name,
    age: age,
    isAdult: age >= 18
  };
}
console.log(createUser("Aman", 20));
```

14.3 Using a callback with array methods

```
const numbers = [1, 2, 3, 4, 5];

const doubled = numbers.map(function (n) {
  return n * 2;
});
const evens = numbers.filter(n => n % 2 === 0);
const total = numbers.reduce((sum, n) => sum + n, 0);

console.log(doubled); // [2, 4, 6, 8, 10]
console.log(evens);   // [2, 4]
console.log(total);   // 15
```

map(), filter() and reduce() are Array methods, not Function methods. They are included because they commonly receive callback functions.

15. Common mistakes

• **Forgetting parentheses:** `greet` refers to the function; `greet()` calls it.

• **Returning vs printing:** `console.log()` displays a value; `return` gives a value back to the caller.

• **Calling a callback too early:** pass `callback`, not `callback()`, when the receiving function should call it.

• **Arrow this confusion:** arrow functions do not create their own `this`.

• **Missing base case:** recursion needs a reliable stopping condition.

• **Using Function constructor:** avoid dynamically building code from strings.

16. Interview quick comparison

Concept	Key difference
Declaration vs expression	Declaration defines a named function; expression creates a function value, often assigned to a variable.
Normal vs arrow	Normal function has call-dependent this and can be constructible; arrow uses lexical this and cannot be constructed.
call vs apply	Both call now; call takes comma-separated arguments, apply takes an array-like argument list.
bind vs call	bind returns a new function for later; call executes immediately.
Callback vs higher-order	Callback is the passed function; higher-order function is the function that accepts/returns functions.
Return vs console.log	return supplies a result; console.log prints for inspection.

17. Practice questions

■ Write a function that checks whether a number is even.

■ Write a function that returns the largest of three numbers.

■ Use rest parameters to calculate the average of any number of values.

■ Create a counter using a closure.

■ Demonstrate call(), apply() and bind() using one object and one function.

■ Write a recursive function to calculate the sum from 1 to n.

■ Create an async function that fetches JSON and handles errors.

■ Explain why an arrow function cannot be used with new.

18. Solutions to selected practice questions

Even number

```
function isEven(n) {
  return n % 2 === 0;
}
console.log(isEven(8)); // true
```

Largest of three

```
function largest(a, b, c) {
  return Math.max(a, b, c);
}
console.log(largest(10, 25, 7)); // 25
```

Average with rest parameters

```
function average(...values) {
  if (values.length === 0) return 0;
  const sum = values.reduce((total, value) => total + value, 0);
  return sum / values.length;
}
console.log(average(10, 20, 30)); // 20
```

Recursive sum

```
function sumTo(n) {
  if (n <= 0) return 0;
  return n + sumTo(n - 1);
}
console.log(sumTo(5)); // 15
```

19. Final reference checklist

✓ Function creation: declaration, expression, arrow, constructor, method, generator.

✓ Core Function.prototype methods: apply(), bind(), call(), toString(), [Symbol.hasInstance].

✓ Key properties: name, length, prototype; legacy arguments/caller noted.

✓ Function concepts: parameters, arguments, return, this, callback, closure, recursion, IIFE.

✓ Modern patterns: rest/default/destructured parameters, promises, async/await.

Remember: Function methods are different from Array methods and String methods. This reference focuses on JavaScript functions and the methods/properties directly related to them.